01 · Browser → Proxy → Stream Server

ESTADO ATUAL · comandos server / proxy · HTTP sem proteção criptográfica

BROWSER
O teu navegador
Tempo: de cima para baixo

PROXY · :8080
ProxySocketServer.run()
Tempo: de cima para baixo

STREAM SERVER · :8081
StreamServer.run()
Tempo: de cima para baixo

0 · Arranque do proxy
main() → ProxySocketServer.run()
ServerSocket(8080).accept()
pool.submit { handle(...) }

0 · Arranque do servidor
main() → StreamServer.run()
Valida a diretoria media/
ServerSocket(8081).accept() → handle()

1 · Abres localhost:8080

GET / →

ProxySocketServer.handle()
HttpUtil.readLine(): pedido + headers
Rota / → servePage(out)
Escreve headers HTTP + INDEX_HTML

← **200 OK + HTML**

2 · A página pede o catálogo

GET /catalog →

relayResponse() → GET /catalog →

StreamServer.handle()
serveCatalog() → Files.list(media/)
HttpUtil.encodeName() → lista de nomes
writeHeaders() + corpo da resposta

← **200 OK + catálogo**

← **relayResponse() → lista para o browser**

3 · Ao selecionar / reproduzir um vídeo

Browser: GET /video/nome.mp4 [+ Range] → ProxySocketServer.handle()
Proxy: relayFromOrigin() → relayResponse() → GET /media/nome.mp4 [+ Range]
Servidor: handle() → serveFile() → writeHeaders() → RandomAccessFile.read() em blocos de 1024 bytes
Regresso: 200/206 + MP4 → proxy lê e reencaminha → browser descodifica e reproduz

02 · Handshake · arranque, Type 1 e envio de Type 2

IMPLEMENTADO · teste TCP separado :8082 · o browser não participa no HandshakeDemo

0 · Cada processo lê apenas o seu próprio .config

main() → HandshakeDemo.proxy(args) / HandshakeDemo.server(args)
CryptoConfigLoader.load(path, expectedRoot) → CryptoConfig + KeyMaterial + HandshakeConfig
CryptoConfigValidator.validate() consulta CipherSuite / MacSuite → ValidatedCryptoConfig
HandshakeConfigValidator.validate() → perfil executável AES-GCM; tudo antes de abrir sockets
Servidor: ServerSocket.accept(); proxy: Socket.connect(); ambos definem socket.soTimeout

PROXY · ProxyHandshake.perform(in, out, media)

Construtor volta a validar o perfil do handshake.
HandshakeWire.validateMedia(media)
SecureRandom: NP (clientRandom) e SEQX
DiffieHellmanSession.createSession(): par DH A/a
exportPublicKeyBytes(): A pública em formato DER
HandshakeWire.proposal(): ProposedCryptoConfig

SERVIDOR · ServerHandshake.perform(in, out)

Construtor volta a validar o perfil do handshake.
Obtém uma cópia local da PSK.
HandshakeWire.read(CLIENT_HELLO)
→ ProtocolMessageCodec.read(input, limite)
Bloqueia à espera da primeira mensagem.
Não usa ainda a chave DH recebida.

1 · Construir e autenticar ClientHello

ClientHelloPayload(SEQX, PCC, A, NP, tag vazia)
HandshakePayloadCodec.encodeClientHelloBody()
HandshakeWire.frame(): MessageHeader + payload
HandshakeKeySchedule.clientHelloAuthentication()
HandshakeCrypto.hmac(chave, header, body)
HandshakeWire.send() → ProtocolMessageCodec.write()

1 · Ler e validar ClientHello

HandshakeWire.read(): versão + Type esperado
HandshakePayloadCodec.decodeClientHello()
HandshakeKeySchedule.clientHelloAuthentication()
HandshakeCrypto.hmac() → MessageDigest.isEqual()
Compara PCC com HandshakeWire.proposal(local).
Valida SEQX; só depois faz trabalho DH.

TYPE 1 · header || SEQX || PCC || NP || A || HMAC-SHA256 →

Type 1: campos em claro; HMAC cobre header + body. A PSK nunca é enviada.

2 · Proxy aguarda ServerHello

HandshakeWire.read(SERVER_HELLO)
→ ProtocolMessageCodec.read()
A resposta tem um prefixo público NB.
NB permite derivar a chave antes de decifrar SRV.
O proxy ainda não conhece B nem o segredo Z.

2 · Preparar ServerHello

SecureRandom: NB (serverRandom) e SRV (desafio)
DiffieHellmanSession.createSession(): par DH B/b
calculateSecret(A): valida A e calcula Z = DH(b, A)
HandshakeCrypto.sha256(frame1) → H1
HandshakeKeySchedule.serverHelloProtection()
PSK + NP + NB + H1 → chave e IV de Type 2

3 · Cifrar e enviar Type 2

ServerHelloPayload(SRV, SEQX+1, CCC, B)
HandshakePayloadCodec.encodeServerHello()
HandshakeWire.frame() reserva o comprimento final.
HandshakeCrypto.encrypt(): AES-GCM + tag
AAD = header2 || NB || H1
HandshakeWire.send() → ProtocolMessageCodec.write()

← **TYPE 2 · header || NB || GCM(SEQY || SRV || CCC || B)**

NP e NB são públicos; SRV e B seguem cifrados. Cada frame inclui um header binário de 9 bytes.

03 · Handshake · validar Type 2, Type 3 e obter a sessão

IMPLEMENTADO até HandshakeSession · integração no vídeo ainda por desenvolver

PROXY · continua ProxyHandshake.perform()

HandshakeWire.read(SERVER_HELLO) devolve frame2.
Separa prefixo NB do ciphertext + tag.
HandshakeKeySchedule.serverHelloProtection()
→ mesma chave/IV que o servidor derivou
HandshakeCrypto.decrypt(AAD = header2 || NB || H1)
Verifica tag GCM antes de devolver plaintext.

SERVIDOR · continua ServerHandshake.perform()

Type 2 já foi enviado.
Tem Z, PSK, NP, SRV e os dois frames.
HandshakeCrypto.sha256(frame1, frame2) → H12
HandshakeKeySchedule.sessionSecret()
PSK + Z + NP + SRV + H12 → segredo da sessão
Apaga a cópia do array Z com fill(0).

4 · Aceitar resposta e calcular segredo

HandshakePayloadCodec.decodeServerHello()
Valida SEQY = SEQX+1 e CCC = proposta.
DiffieHellmanSession.calculateSecret(B) → Z
HandshakeCrypto.sha256(frame1, frame2) → H12
HandshakeKeySchedule.sessionSecret() → segredo
Apaga a cópia do array Z com fill(0).

4 · Aguardar confirmação do proxy

HandshakeWire.read(CLIENT_FINISHED)
→ ProtocolMessageCodec.read()
HandshakeKeySchedule.clientFinishedProtection()
segredo + H12 → chave/IV próprios de Type 3
Não começa a transmitir media nesta fase.

5 · Construir e cifrar ClientFinished

ClientFinishedPayload(SEQX+2, SHA256(SRV), media)
HandshakePayloadCodec.encodeClientFinished()
HandshakeKeySchedule.clientFinishedProtection()
HandshakeWire.frame() → header3
HandshakeCrypto.encrypt(AAD = header3 || H12)
HandshakeWire.send() → ProtocolMessageCodec.write()

5 · Verificar a confirmação recebida

HandshakeCrypto.decrypt(AAD = header3 || H12)
HandshakePayloadCodec.decodeClientFinished()
Valida SEQZ = SEQX+2.
MessageDigest.isEqual(R, SHA256(SRV))
HandshakeWire.validateMedia(media)
Autenticação GCM confirma posse do segredo DH.

TYPE 3 · header || GCM(SEQZ || SHA256(SRV) || MEDIA) →

6 · Em ambos os participantes: construir HandshakeSession

HandshakeCrypto.sha256(frame1, frame2, frame3) → H123 (transcript completo / sessionId)
HandshakeKeySchedule.trafficKeys(segredo, H123) → SessionKeys(proxyToServer, serverToProxy)
Cada direção tem ProtectionKeys(key, iv); KeyMaterial guarda cópias e oculta segredos em toString().
HandshakeSession(keys, media, firstMediaSequence = SEQX+3, sessionId = H123)
HandshakeDemo imprime apenas media / ID público, fecha o socket e termina o teste.

Confirmação vista por cada lado

Servidor: Type 3 verificado → aceita a sessão. Proxy: Type 2 verificado + Type 3 enviado.
O proxy só terá confirmação da aceitação pelo servidor ao receber um primeiro registo protegido válido.

PRÓXIMA INTEGRAÇÃO · percurso pretendido, ainda sem implementação

Browser → pedido HTTP ao proxy → handshake com o servidor → Type 4 (blocos protegidos) → proxy
Proxy verifica sequência/tag, decifra e entrega bytes MP4 ao browser → Type 5 (END autenticado).
Falta definir/implementar contadores de nonce, registos Type 4/5, Range/catálogo e métricas.
O IV de tráfego derivado aqui é uma base: não se pode reutilizá-lo diretamente em todos os blocos.

04 · Do objeto Kotlin aos bytes no socket

Camadas chamadas pelos coordenadores ProxyHandshake / ServerHandshake

ENVIO · do significado para a rede

1. DTO: ClientHelloPayload / ServerHelloPayload / ClientFinishedPayload + ProposedCryptoConfig
   Guarda campos com significado; ainda não é a trama.
2. HandshakePayloadCodec.encode...()
   DataOutputStream → ByteArrayOutputStream
   writeLong / writeInt / writeText / writeFixed / writeSized; writeConfig() só envia dados públicos.
3. HandshakeKeySchedule + HandshakeCrypto
   Type 1: HMAC(header || body).
   Types 2/3: encrypt(plaintext, key, iv, AAD).
4. HandshakeWire.frame() → ProtocolMessage
   MessageHeader + payload final (ByteArray).
   O header final é preparado antes de HMAC/AAD.
5. HandshakeWire.send(output, frame, limite)
   → ProtocolMessageCodec.write()
   → encodeHeader() → write(header) + write(payload)
   → flush() no OutputStream do socket.

RECEÇÃO · da rede para o significado

1. InputStream do socket
   TCP entrega um fluxo: uma leitura pode ser parcial.
2. ProtocolMessageCodec.read(input, limite)
   Lê 9 bytes de header; extrai tipo e tamanho.
   Valida tamanho antes de alocar; lê payload todo.
3. HandshakeWire.read(input, esperado, limite)
   Valida versão/release e Type esperado.
   Type 255 recebido → aborta.
4. O coordenador aplica autenticação / cifra
   Type 1: decode limitado → verificar HMAC.
   Types 2/3: decrypt() verifica a tag GCM
   antes de disponibilizar o plaintext ao decoder.
5. HandshakePayloadCodec.decode...()
   DataInputStream: readLong / readInt / readText / readFixed / readSized + readConfig().
6. Coordenador verifica PCC/CCC, sequências, desafio e DH conforme a etapa.

TCP · bytes ordenados; não preserva fronteiras de mensagens

TRAMA NO FIO · big endian

version: 2 bytes | release: 2 bytes | type: 1 byte | payloadLength: 4 bytes | payload: N bytes
Header = 9 bytes. payloadLength inclui prefixos públicos, ciphertext e tags, quando existirem.
Os DTOs, a PSK e os ficheiros XML não são serializados automaticamente para a rede.

SE ALGO FALHAR · localizar a camada primeiro

XML mal formado / tag em falta → CryptoConfigLoader.load(); nem chega a abrir o socket.
Combinação / perfil não suportado → CryptoConfigValidator / HandshakeConfigValidator.
Recusa de ligação / timeout → socket.connect(), accept() ou leitura com soTimeout.
Trama truncada / tamanho inválido → ProtocolMessageCodec.read(); tipo/versão → HandshakeWire.read().
HMAC inválido (Type 1) → PSK, chave derivada ou header/body diferentes.
Tag GCM inválida (Types 2/3) → chave, IV, AAD ou ciphertext diferentes / adulterados.
Tag válida mas sequência / CCC / desafio incorretos → verificações do coordenador.
catch: HandshakeWire.alert() tenta enviar Type 255; relança erro; Demo.use fecha a ligação.

Type 255 antes da sessão é diagnóstico não autenticado; não autoriza baixar a segurança.
Este mapa descreve o código atual de 26/10/2026; atualizar ao integrar o streaming protegido.